

Developer Operations (devops)

Liveness / Readiness



In-Place Pod Resize Graduated to Beta in k8s v1.33

We discussed VPA:

- Resizing a Pod requires recreation
- Pod spec container resources = immutable

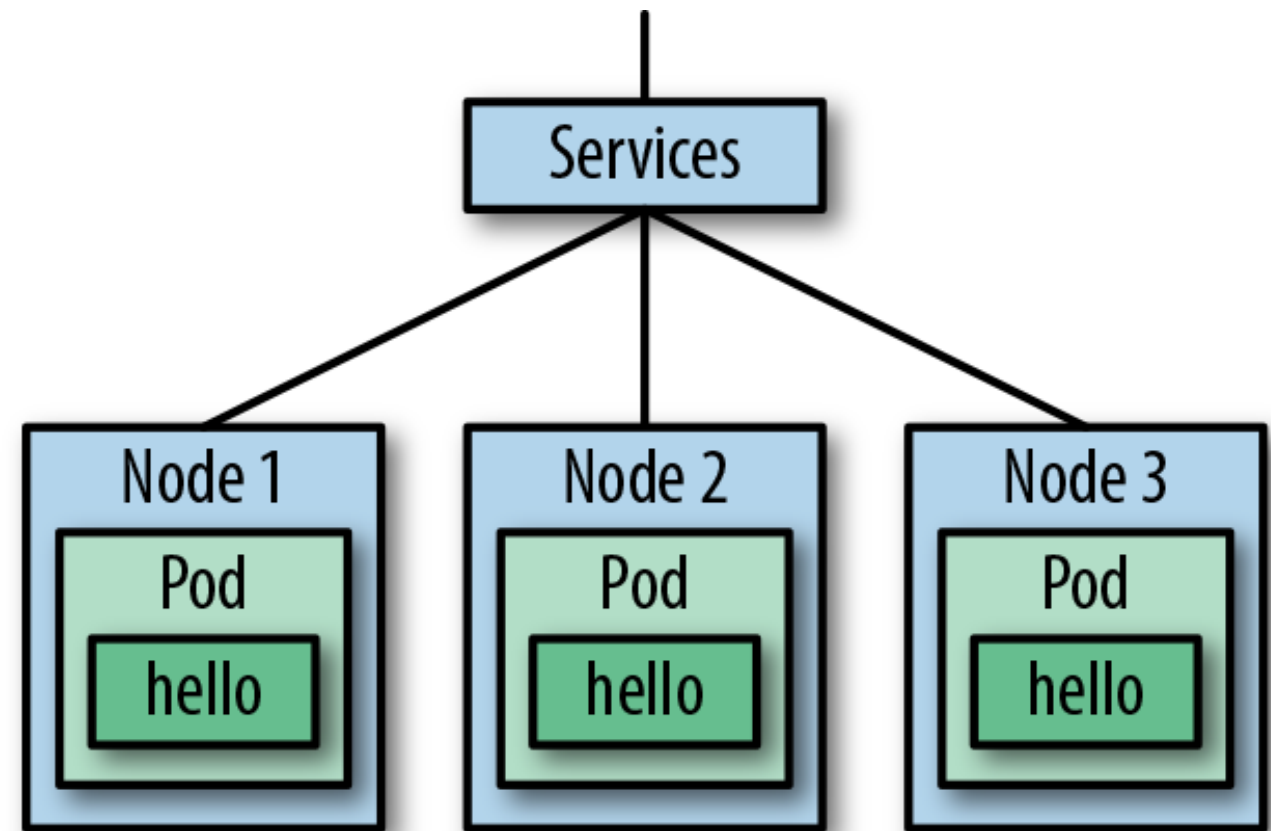
News with k8s v1.33

- <https://kubernetes.io/blog/2025/04/23/kubernetes-v1-33-release/#beta-in-place-resource-resize-for-vertical-scaling-of-pods>
- <https://kubernetes.io/docs/tasks/configure-pod-container/resize-container-resources/>
- change the CPU and memory resource requests and limits ***without recreating the Pod***



[Recap] How to access Container? Service

What are your experiences with the chatbots
you have regarding starting the pod and
serving traffic instantly?



How to you get awareness, when an application runs “normal”?

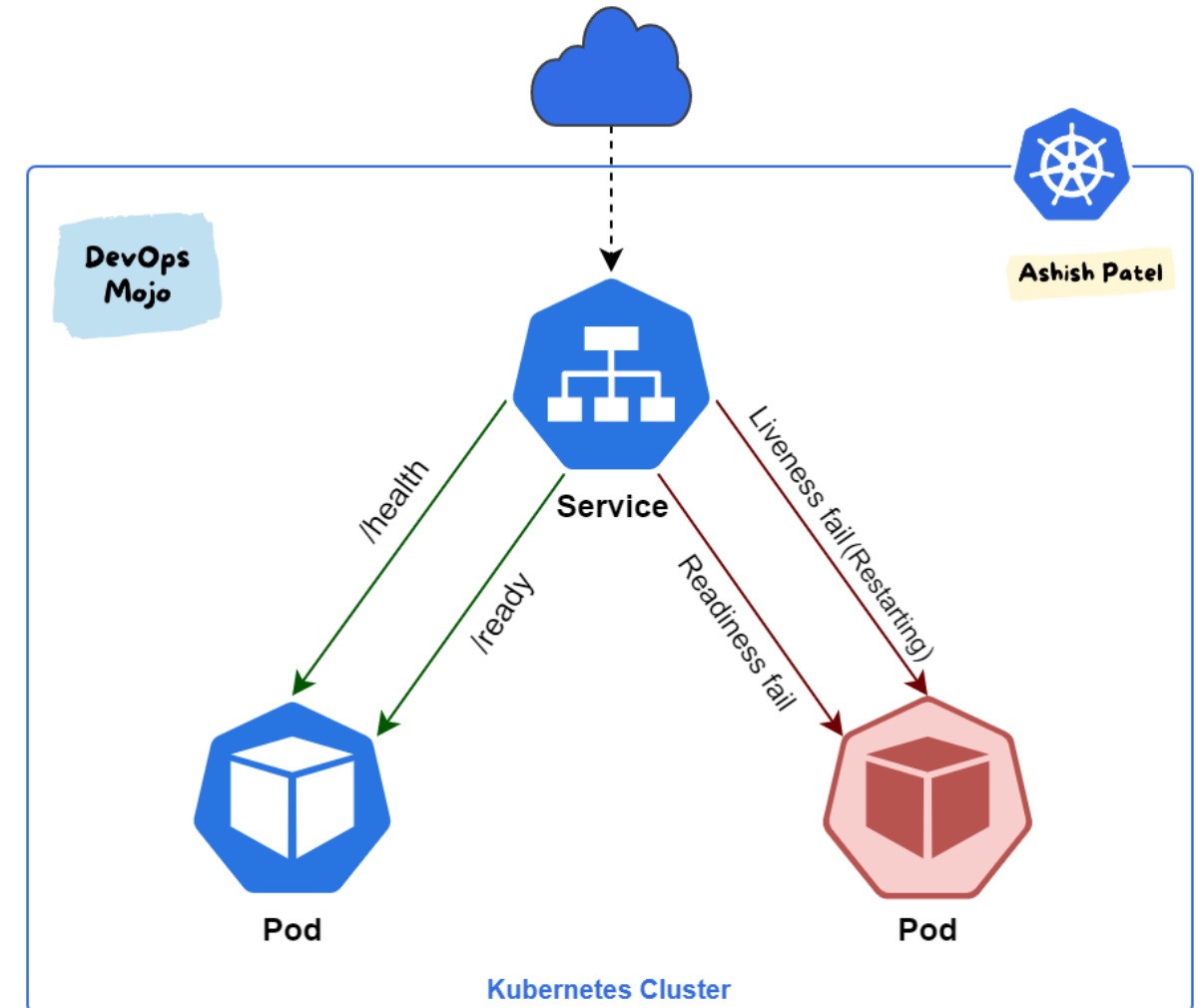
Liveness and Readiness

Liveness:

- is Pod alive?
- Restart Pod if Pod is generating failure
- If not specified, PID-handler is active

Readiness:

- Is Pod able to handle traffic?
- Disables traffic forwarding through service
- If not specified, PID 1 in the container is enough



How to implement it?

```
@app.route('/ready')
def ready():
    if redis_store.ping():
        return "Yes"
    else:
        flask.abort(500)
```

Framework support

- Check documentation of framework to get direct support

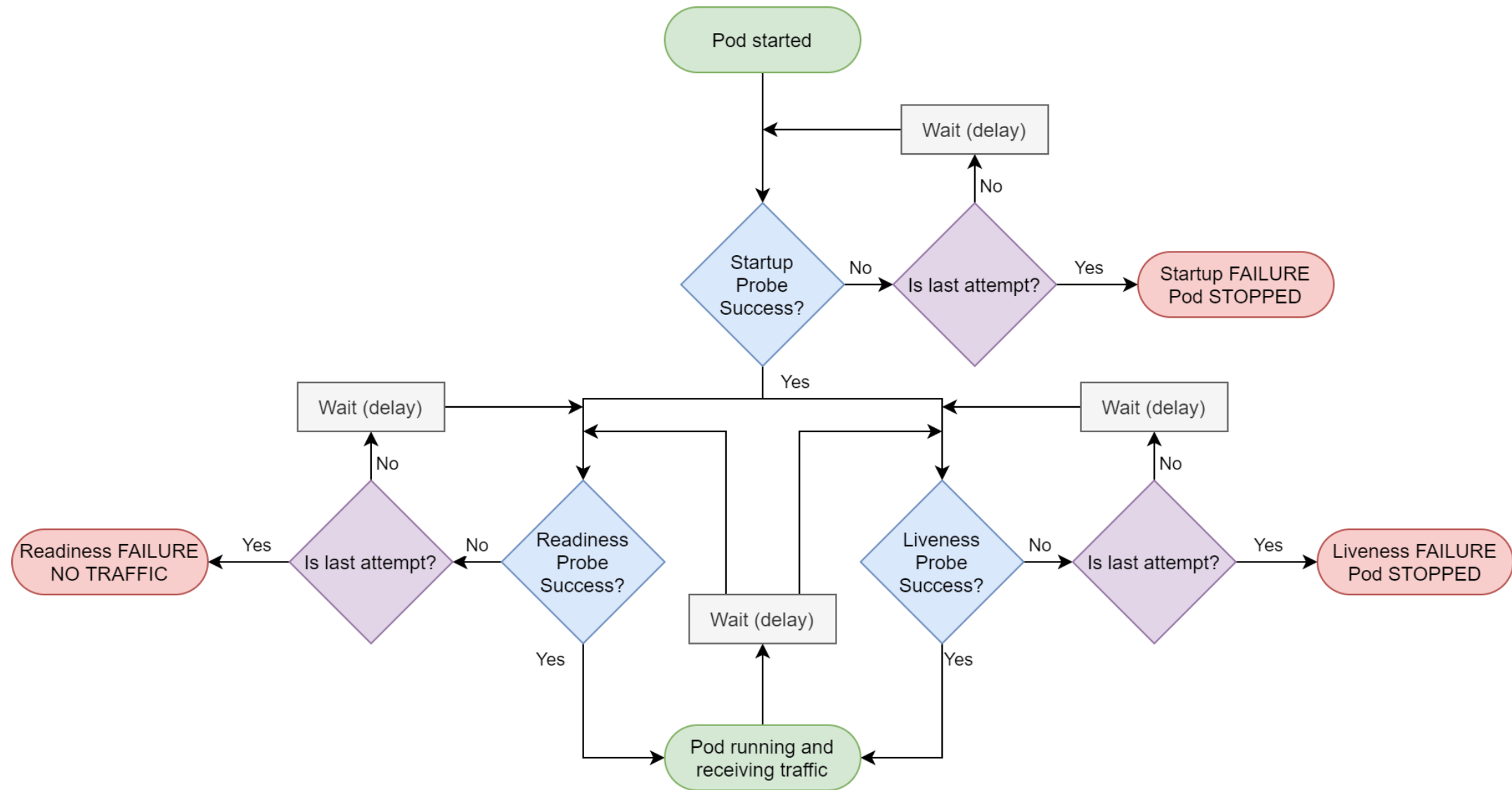
Custom endpoints:

- <https://ahmet.im/blog/advanced-kubernetes-health-checks/>

However, probes must respect the application.

```
apiVersion: apps/v1beta1
kind: Deployment
metadata:
  name: flask
  labels:
    run: flask
spec:
  template:
    metadata:
      labels:
        app: flask
    spec:
      containers:
      - name: flask
        image: quay.io/kubernetes-for-developers/flask:0.3.0
        imagePullPolicy: Always
        ports:
        - containerPort: 5000
        envFrom:
        - configMapRef:
            name: flask-config
        volumeMounts:
        - name: config
          mountPath: /etc/flask-config
          readOnly: true
        livenessProbe:
          httpGet:
            path: /alive
            port: 5000
            initialDelaySeconds: 1
            periodSeconds: 5
        readinessProbe:
          httpGet:
            path: /ready
            port: 5000
            initialDelaySeconds: 5
            periodSeconds: 5
```

Workflow Readiness, Startup and Liveness

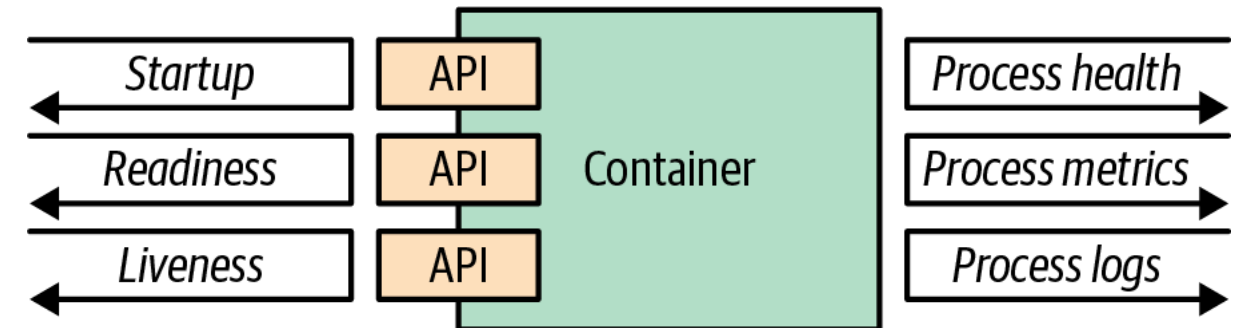


Bigger Picture

Health is more than just

Startup/Readiness/Liveness:

- Process Handling: Keep track of different Signals
- Metrics: Get awareness about metrics of pod (CPU, RAM, Business Metrics)
- Logs: Structured Logging for centralized parsing / querying



Demo

The screenshot shows a VS Code editor with a project named 'roberta'. The file explorer on the left shows the project structure, including 'src/roberta' with files like 'app.py', 'cadettiny.py', and 'config.py'. The main editor displays the 'app.py' file, which contains the following code:

```
def create_app(test_config=None):
    def send():
        if response:
            metric_requests.labels('true').inc()
        else:
            metric_requests.labels('false').inc()
        return response

    @app.route('/livez')
    def liveness():
        return Response(response="OK", status=200)

    @app.route('/readyz')
    def readiness():
        if cadet_tiny.loading_complete():
            return Response(response="OK", status=200)
        else:
            return Response(response="NOK", status=400)

    return app
```

The bottom panel shows the output of the command 'flask run', indicating that the Flask app is serving on '0.0.0.0' and '127.0.0.1'.



<https://github.com/sebinxavi/kubernetes-readiness/tree/master>

→ Other example

